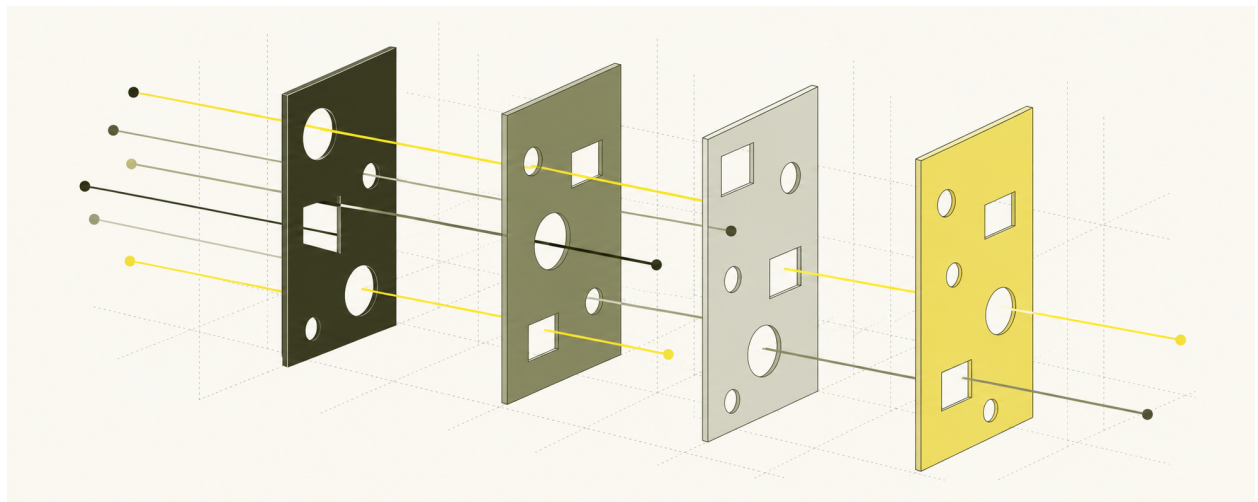


# Choosing and Routing Open Safety Models: A Practitioner's Guide

*Contributors:*

*Musubi: Alice Hunsberger, Juliet Jonak, additional research by Nikki Marinsek*

*ROOST: Jeremie Ponak*



The Swiss Cheese Model for Safety – originally formally suggested by James T. Reason. This image was generated with the help of ChatGPT Astra.

# Contents

|                                                                                  |           |
|----------------------------------------------------------------------------------|-----------|
| <b>Intro</b>                                                                     | <b>3</b>  |
| <b>Part 1: On model selection</b>                                                | <b>4</b>  |
| Accuracy                                                                         | 4         |
| Price & Size                                                                     | 6         |
| Latency                                                                          | 7         |
| Steerability                                                                     | 7         |
| Other considerations for steerability                                            | 9         |
| Other dimensions for robustness: across languages and under adversarial pressure | 9         |
| <b>Part 2: Routing or Cascading between Models</b>                               | <b>10</b> |
| Route/ cascade policies and models if:                                           | 10        |
| Use a single model if:                                                           | 11        |
| Best practices for routing                                                       | 11        |
| Reference routing architectures                                                  | 14        |
| Combining architectures: live chat                                               | 17        |
| <b>Conclusion</b>                                                                | <b>18</b> |
| <b>Appendix 1: Examples of Routing Architecture</b>                              | <b>19</b> |
| <b>Appendix 2: Additional Routing Model Examples</b>                             | <b>24</b> |

# Intro

In 1996, an entire town in northern England couldn't sign up for AOL. Residents' registrations kept getting rejected by a profanity filter because a four-letter word hid in their town's name: Scunthorpe. The mistake became known as the [Scunthorpe Problem](#) and represented state-of-the-art moderation at the time.

Thirty years later, our shared internet spaces have multiplied and thankfully, safety teams have more sophisticated options to choose from when protecting their users from harm (even if broad keyword filtering still has many good use cases).

One option is proliferating quickly: models designed primarily for safety. But choosing the right model for the job hasn't gotten easier. Capacity strapped teams may feel overwhelmed when looking at the options available on [Hugging Face](#) or the [ROOST Model Community](#).

We're all trying to avoid becoming a Scunthorpe 2.0: a model that doesn't follow your community's policies, or one whose cost and speed only let you review a fraction of your content. So how do you choose the right model?

This guide makes that choice easier. Whether you're new to models or already utilizing them, each section below will help break down one factor in the process.

No single model is best for safety work. The right pick depends on the job, where the model sits in your product, how well it follows your policies, and whether its cost and speed let you review the right amount of content.

The guide is broken into two parts:

**Part 1:** How to narrow down models to a few worth experimenting with, using questions like:

- How accurate does your model need to be? What does a miss cost, and what does a false flag cost?
- What's your budget and scale?
- How fast does a decision need to come back?
- How often does your policy change?
- Which languages do you support?
- How much adversarial pressure do you expect?

**Part 2:** How to combine models together effectively. Often, a single model won't work for every decision you need to make. Different surfaces and harms have varied requirements for speed, cost, policy fidelity, and review.

We share reference architectures, such as single-pass systems, cascades, and routing by surface or harm. We'll also cover how to set thresholds and escalation paths between models. By the end, you will be able to design a moderation stack that best fits your team's actual policies and operating constraints.

## Part 1: On model selection

There are a lot of ways to compare moderation models, but they are not all equally useful. We recommend starting with these four:

1. How accurate is it on your policy?
2. What does it cost at your scale?
3. How fast is it?
4. How easily can you steer it as your policy changes?

Those four usually determine whether a model is viable at all.

From there, the rest depends more heavily on your product: language coverage, adversarial robustness, deployment requirements, and the kinds of outputs or explanations you need. A model can look excellent in the first four categories, but make sure that these other considerations don't leave gaps in your coverage or use-case.

### Accuracy

Our journey to choosing the right model starts with accuracy: you want a system that will, with some degree of certainty, capture content that you want removed. You also want to make sure the model doesn't overflag benign content, for reasons ranging from added noise to genuine cost concerns. No model will be perfect, but you should know which errors the model makes, how often it makes them, and whether those errors are acceptable for the job you want it to do.

Public benchmarks are useful for narrowing the field. [Artificial Analysis](#) compares guardrail models across F1, recall, specificity, latency, and price across three public harms datasets (WildGuardTest, ToxicChat, and XStest), while [RabakBench](#) gives a more localized view of moderation performance across Singlish, Chinese, Malay, and Tamil. Together, they are useful for spotting broad performance differences and language-specific gaps, but neither can substitute for evaluation on your own policy and traffic. For that, you'll need to run the model on your own data, and peer into the model's performance to determine what works best for you.

### A glossary of model accuracy

**Recall:** Of all the content that actually violates the policy, how much did the model catch?

**Precision:** Of everything the model flagged, how much was actually a violation?

**F1 score:** A single score that balances precision and recall. Higher is better when you care about both.

**False positive:** Safe content that gets flagged & **False negative:** Violating content that gets missed.

So to test accuracy, depending on what you're using the model for, you'll want to prioritize different components of an accurate model:

- **Focus on recall for a first pass that routes harmful content.** It's a net: the goal is to catch as much potentially violating content as possible, even if that means sending some safe content onward.
- **Focus on precision for a second layer or deeper policy review.** At this stage, false positives are more expensive because they can trigger enforcement, human review, or user friction. You want the model to be more confident that something really violates the policy before acting.
- **Balanced F1 for a model that decides on its own.** When there is no later stage to correct errors, you need a reasonable balance between catching violations and avoiding false flags.

Once you know which measure of accuracy matters most, consider what you need the model to return. A classifier can give you a score that you can threshold or use for routing, while some generative models can produce a reason alongside the decision. If a human needs to review or understand the decision, that output format may be helpful.

### What should your model output?

**Extra outputs affect cost and consistency.** In Musubi's testing, open-ended fields like a step-by-step analysis or general image description were most useful during policy iteration, but also increased how much threshold scores varied between runs.

If a moderation team doesn't need these fields in production, Musubi could reduce output tokens by ~83%, with no loss in recall. Threshold scores moved about two-thirds less.

**Targeted fields behave differently.** Asking the model to answer a specific question before making a decision improved accuracy and consistency (e.g. "Is an adult's face visible?").

The next step is testing on your own data. Most teams start by building a golden set from examples they've reviewed and agreed on. This exercise exposes inconsistent labels or vague policy language. Musubi has found that reviewing disagreements prior to tuning can reduce a policy's misses by more than half, with no increase in false positives.

Then:

- Calibrate thresholds on your own data before go-live.
- Run in shadow mode first, then hand-sample production decisions so you can separate model errors from genuine policy grey areas.
- Stress-test the boundaries. Add synthetic examples for tricky categories, exceptions, and near-boundary cases, using your real examples as the template.
- Test the whole workflow, not just each model. Include multi-violation examples, safe look-alikes, and a realistic share of violating content.

## Price & Size

What a model costs sets how much traffic your systems will review. A cheap model may be viable on every message or upload, while a more expensive one may only make sense for a sample or for escalated cases.

Classifiers ([BERT](#)-family models like [RoBERTa](#) or [DistilBERT](#), and other lightweight models) read the text once and output a score or label directly, making them inexpensive. Generative models on the other hand produce their answer word by word, and every word is another pass through the model, which is where most of an LLM's cost and latency come from. A new middle ground is decision models, like [Laya](#) and [PolicyLM-1.7B](#). These are small models that return scores for predefined answers rather than generating a written response token by token, so they're fast and cheap to run like classifiers, but allow you to customize your taxonomy and policies like an LLM.

Within a model family, cost scales with size: a mixture-of-experts model activates a fraction of its total parameters on each pass, and a quantized model runs smaller than its full-precision weights. Run size also determines the compute you need, with a 1–2B parameter model fitting easily on a CPU, while a 20B+ model, like [gpt-oss-safeguard](#), needs dedicated inference hardware.

For safety teams, the decision hinges on finding the right amount to spend, on which model, on which part or problem of their organization. It's a placement question that answers: “where does a small, cheap model work, and where do I need better precision (and hence should spend more)”. We talk more about this in part 2 of this guide.

*Musubi learned this firsthand: downsizing the routing model while keeping the specialist stages on a larger model cut cost materially, raised escalations to the specialist model slightly, and didn't reduce recall.*

Finally, when you're evaluating the cost of the model, consider the downstream human-review cost as well. Auto-approving clean content is often the biggest operational saving, even when the per-call price is higher!

## Latency

Price sets how much traffic your model will be able to cover, and latency will set which surfaces you can cover at all. These are two sides of the same coin, and often somewhat of a tradeoff: the models fast enough for live surfaces are usually the cheap ones, and the models worth escalating to cost more per call.

The best approach when it comes to resolving this tension is to set the budget by surface. In live chat, game lobbies, DMs, and usernames, the decision has to land before the conversation moves on, which means tens to low hundreds of milliseconds. Historically, live chat has only ever had fast classifiers. Generative LLMs take seconds per check, and it grows with the length of the answer. That's also why purpose built decision models are often useful: PolicyLM-1.7B by Musubi for example is a small enough model to remove latency, but that can also read your policy at classifier speed. More on why that's important when we talk about steerability.

Other cases don't have the same level of requirements for review: uploads, profiles, and listings, a second or more is usually fine. Async and batch review can take minutes to hours. This all depends on your user, and the industry.

Model time alone isn't the only consideration for latency, you'll also want to measure the entire system time. For example, a model that only takes 80ms to run on average can still produce a 600ms decision if it sits behind an API – which may or may not be tolerable depending on the moderation surface. For a live chat, it's another reason why a single-pass model is usually the best approach.

## Steerability

Steerability is defined across two categories. The first is **output control**: can you define the categories the model returns, or are you stuck with someone else's taxonomy? The second is **policy fidelity**: once you give the model your rules, how faithfully does it follow them—including exceptions and carveouts?

A fixed classifier like Meta's LlamaGuard offers little of either. PolicyLM gives you output control and substantial policy fidelity, though it is less reliable when a policy asks it to reverse an expectation learned during training. [Zentropi's CoPE](#) is designed to support both.

Let's say you're moderating a dating site. Flirting and sexual talk between consenting adults is the point of the product, so a fixed "sexual content" category might flag the conversations you want to allow. Meanwhile, the harms you care most about, like a match pushing to move the chat to WhatsApp right away or asking for gift cards, don't appear in a generic taxonomy at all. A steerable model would let you write that policy down: consensual flirting is fine, unsolicited explicit images aren't, and early requests to move off platform get flagged.

ROOST has written about steerability before (see our [Shieldstral](#), [CoPE-B-A4B](#), and [gpt-oss-safeguard](#) posts), so we'll keep it short here. A steerable model is only as good as the policy you give it (the ROOST Model Community's [policy packs](#) are a good place to start). For a public measure, [Zentropi's Policy Steerability Benchmark](#) is the cleanest: it gives a model the same content under two policies, one that should flag it and one that should permit it, and scores the fraction of pairs where the model follows both.

Steerable doesn't mean that the model will be perfectly deterministic. The same policy can behave very differently across sizes of the same model family and versions: one flip-flops on hard categories, one is consistently permissive, one is consistently strict.

Each model has its own personality and quirks, so test each one against your golden set even with an identical prompt, and expect to tune the policy per model, since identical prompts won't get the best out of each. And the newest versions of models won't always mean that they're better tuned to following your instructions. We regularly test across model families, isolating differences due to model size, reasoning and temperature settings.

*In one example, identical policy verbiage gave different results when evaluating whether a minor was accompanied by an adult. Gemini 2.5 Flash and 3.5 Flash-Lite outperformed 3.1 Flash-Lite, which reached for anything that could plausibly justify a child as accompanied: a hand at the edge of the frame, an adult walking in the distance, or a group of children accompanying one another. This specific model version required additional definition to produce consistent results.*

The best way to test this is similar to testing the accuracy of a model: run examples on your own policies, invert the model, make carve outs and see if the model listens, change the topic. For more examples of how to easily run these kinds of tests, ROOST's Director of Product Juliet Shen has built small sample datasets and libraries inside of an evaluation harness called [Vibecheck](#).

## Other considerations for steerability

Steerability is not only about labels and policy text. A few design choices can materially change how consistently a model applies a rule:

- **Observations:** Some decisions hinge on a single fact. Ask the model to commit to that fact first—for example, `adult_present: yes | no`—before deciding whether a policy is violated. In one Musubi child-safety test, this reduced disagreements with moderators by ~85%.
- **Violation names:** Labels carry meaning for models. More precise names can improve accuracy: “Teen” worked better than “Young Adult,” and “Unaccompanied Minor” better captured the policy intent than simply “Child.”
- **Moderation action:** Design labels around what happens next. A broad “Minor” flag may be too uncertain for automation, while more specific categories such as “Pre-Pubescent” and “Teen” can support different paths for automated action, age verification, or moderator review.

## Other dimensions for robustness: across languages and under adversarial pressure

There are many more dimensions to analyze open safety models through — of which we'll spend a little more time on two: multilingualism and robustness. These are not universal considerations — many models you choose to use may be fine in a single language, and face limited adversarial pressure. But if you're using the model in a setting where adversarial abuse could occur (for example, inside a chatbot where a user is trying to get around guardrails), or where speech could occur in other languages, you'll want your bases covered.

**Multilingualism:** “Multilingual” on a model card doesn't mean your policy performs equally well in every locale, so pressure-test per language, and separate capability from calibration. Machine translation can also move scores by itself (a euphemism becomes explicit in translation), so use native-reviewed test sets when you can, and language-specific allow and block lists are a cheap patch for terms the base model under-catches.

**Robustness:** How much time to spend here depends on the stakes. If the cost of adversarial pass-through is severe, say a model gating CSAM generation, spend real time on it; if you're monitoring message boards, you probably don't need to. Adversarial robustness is whether content can talk the model out of a decision, and the attack to test first on any policy-reading model is policy injection, fake policy text inside the content.

*In ROOST tests on PolicyLM for example, a forged "policy update" cut recall roughly in half, and injected text placed before the content did more damage than text placed after it.*

## Part 2: Routing or Cascading between Models

As you've just seen, not every model is fit for every purpose. Sometimes the best option is to route your content from one model to another in a cascade. In this section we'll discuss when to route between models, considerations to make when routing, and what different routing/cascading architectures might look like in practice.

Not every type of moderation is a good candidate for full automation. Today, it's realistic to use a mix of classifiers and LLMs to automate profile and bio moderation, user-uploaded images, listings and marketplaces, creator uploads, and spam and scam signals on structured events. Policy-aware live chat is newly realistic due to new decision models like Laya and PolicyLM-1.7B: until recently the only option at that speed was a fixed classifier. Agent-to-agent safety, video (with sparse frame sampling), audio, and AI generated content identification are emerging but often viable. A human should always be in the loop for high-severity, high-risk categories such as reports of offline harms.

When deciding whether to use a single model or route between models, consider the content that is being moderated, the policies or harm types involved, latency and cost restrictions, how much volume you're dealing with, and the amount of complexity you're comfortable with.

### Route/ cascade policies and models if:

**Most of your platform's content is obviously safe.** Use a small, cheap model for the first stage to clear any obviously safe content. Then use a more expensive model (often a reasoning model) to look more closely at anything that might be a violation. It's important to measure the percentage of non-violative content before going live, because the amount of violations sent to your second model is directly correlated to your overall cost. If 20% or less of your content is potentially violative, this is a good choice.

**Your taxonomy has hard subcategories that benefit from a specialist** (minor presence, CSAM-adjacent, self-harm). Reporting workflows often use these policy categories already which can help funnel content to different models and policies.

*In Musubi's testing, one policy with many categories on a capable model performed as well as splitting it up. Splitting only added policies to maintain.*

Smaller models are different. As a policy grows, they start missing subtle categories first, while severe categories hold up. Split out or escalate only those subtle categories.

**Your budget can't cover a large model on 100% of traffic.** Choose where a false positive or lower accuracy really hurts, and route only content in those areas. Everything else goes through a smaller model.

**A minority of decisions needs a written rationale.** Route only those to a model which will output a reason or analysis; the rest go through a model that merely labels/ classifies.

## Use a single model if:

**Decisions have to come back immediately in one pass** (i.e. live chat). You can still dig into serious labels for further investigation later, but tight latency requirements will mean this has to be done asynchronously, not live.

**Your volume is low enough that operational simplicity beats the savings.** Use one good, general-purpose model in these cases.

**Policies are tightly coupled**, so a decision needs reasoning across categories a router would split.

**Your team doesn't yet have the golden-set discipline** to tune handoffs between stages safely.

## Best practices for routing

Teams often start with low-complexity workflows that give them easy wins, then build from there. When deciding how (or whether) to route, decide your latency budget and the cost of a miss first. All other decisions will flow from here. For example, if you have to make decisions extremely quickly, you don't have the option of routing to a reasoning model. Or if you want to avoid a missed violation at all costs, then focus on extremely high-recall before anything else.

**Tune the first pass for recall.** In any tiered workflow, whatever the first pass clears is never looked at again. Its false flags only cost an extra call to the next stage, so err toward flagging. In Musubi's testing, nearly all misses traced back to the first pass rather than later stages; a specialist clearing an item and dropping a second flag was rare. A first pass that returns a single label also can't carry a second violation forward at all.

*Musubi has found that utilizing a high-recall first-pass policy reduced routing costs by more than half, while only increasing moderator manual reviews by 2.5%.*

**Verbiage matters.** Two additional changes kept results consistent, even on smaller models: removing exceptions from the first-layer and providing the context that a miss costs more than an over-flag.

**Route models based on output scores and thresholds where you can, not just labels.** A classifier tells you which category fired (the label) and how confident it is (the score). Check what the model actually returns before you design the routing:

- **A label.** [CoPE-B](#) returns a single 0/1 token for a label, no score.
- **A score per question.** [Shieldstral](#) answers one yes/no policy question per call, with a score you can threshold.
- **A score per label.** [PolicyLM-1.7B](#) returns a 0–1 score for multiple labels in one combined pass.
- **A label with reasoning.** [gpt-oss-safeguard](#) returns a label with a reason. Route on the label; use the reason for review and appeals.

**Return a list of violations, not a single label.** A single label catches violating content just as well, but it can't report a second violation. On smaller models, asking for a score per category makes the model consider each category, and it also gives you a threshold per category.

*When using labels and scores to route between models, labels should decide where an item goes, whereas the score decides whether it goes anywhere.*

**If you have a score on output, you can use it to tune how much gets sent to escalations.**

- Below a low threshold, allow.
- Above a high threshold, act automatically (block, hide, mute).
- In between, escalate to a larger model or a human. This is your cost dial. A wider band means more escalations and more cost, but fewer errors reaching users.

**Be sure to log every score, not just the label.** Scores let you test what different thresholds would look like without having to rerun traffic. Score distributions are also an early warning for drift. If you start to see fewer high-confidence scores, that's a sign to dig into things further. Finally, thresholds should not be set uniformly across the board. You want to adjust them for different scenarios to achieve better performance.

**Check your score distribution before you design an escalation band.** Some models almost never return middling scores. With those, an “unsure” band ends up escalating nearly every flag and waving everything else through, so it doesn't work as a cost dial. With generative models, you can shape that distribution in the policy itself, so the score's meaning is clear for setting automation thresholds.

For example, some platforms treat anyone of ambiguous age as a possible minor. When the policy says to flag ambiguous cases, the model flags them and puts its doubt into the score. This can be a problem if a clear violation and an ambiguous image return the same high score.

*Musubi found adding guidance on which situations belong at each end of the scale, tied to what's observable in the content, made score results more consistent. Top scores became reliably clear violations, ready for automated handling or high priority escalation.*

**Set different thresholds based on severity.** Classifiers often come with pre-set confidence thresholds set for each category to make this easier. Typically, there's a lower threshold for rare, severe harms so you escalate early, and a higher threshold for low-severity harms like profanity so you have fewer needless escalations. For example, [Roblox's PII classifier](#) follows this pattern by recommending a lower threshold (0.10) for the higher risk of steering users off-platform, and higher thresholds (0.60 and 0.55) for the lower risks of asking for and disclosing PII.

**Set different thresholds based on language.** Model performance often differs between languages. For languages with lower performance, set a wider middle escalation band. For models with higher performance, you won't need to escalate as much to a bigger model or human review.

**Set threshold by position in the stack.** A first pass in a cascade should be tuned for recall so it casts a wide net. The model it passes content to next should be tuned for precision, so it dismisses the false positives and goes deeper into policy nuance. In contrast, a model that decides on its own should be tuned for a balance between recall and precision. If you use a recall-tuned model on its own, it floods the queue. If you use a precision-tuned model on its own, it misses too much.

**Set threshold by model.** Different models will return different values. It's important to know that scores aren't comparable across models. A 0.7 from one model isn't a 0.7 from another. Recalibrate when you swap models, and when you edit the policy text.

**Count the policies you have to keep in sync.** Every router, screen, and specialist is another text that has to agree with the others, and upkeep grows with each one. When you add a category, update the router or screen too, not just the new specialist. Old carve-outs in the router can quietly block the new category before it ever reaches the specialist.

**Consider as much information as you can.** Routing rules can mix a classifier score with context the model doesn't see, like account age, prior enforcement, or surface (DM vs. public post). For example, you can escalate a middle-band score from a new account, but allow the same score from a long-standing one.

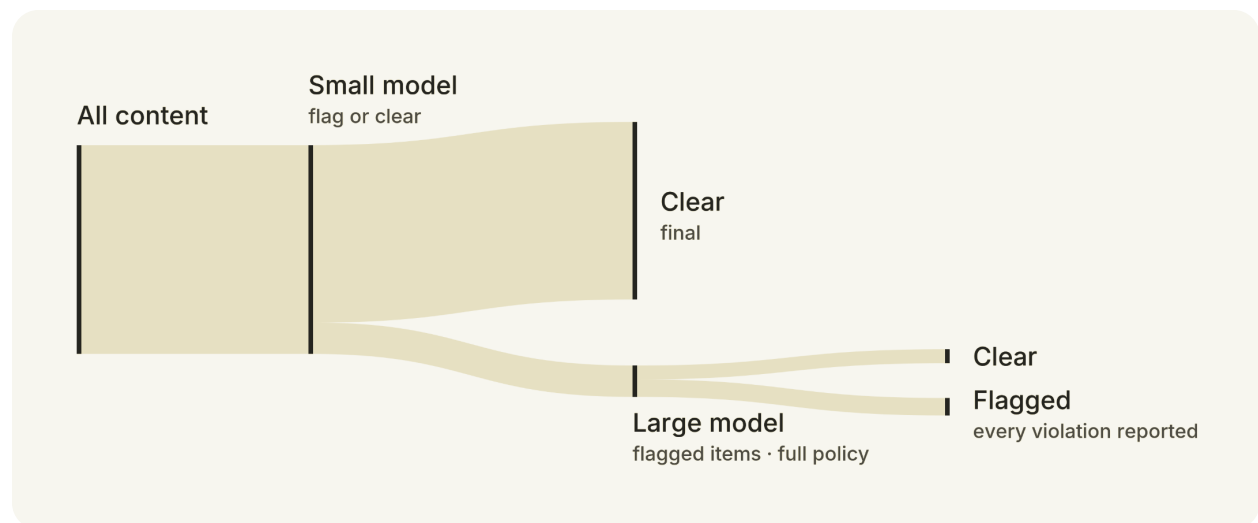
**Create systems to keep track.** The more routing complexity your organization has, the more important it is to have systems and software in place to help you manage the various components and oversee what's happening in production. For example, you want to make sure that you version policies and routing rules, so you can quickly fallback to a prior version if you need to. It's also best practice to run evaluations on sampled production data on a regular basis to look for new errors or changes (drift).

## Reference routing architectures

The following four architectures cover most stacks. They differ in what the first stage returns, which is the decision that shapes everything downstream. Appendix 1 applies them to specific use cases, and Appendix 2 collects the remaining patterns.

### Screen then escalate

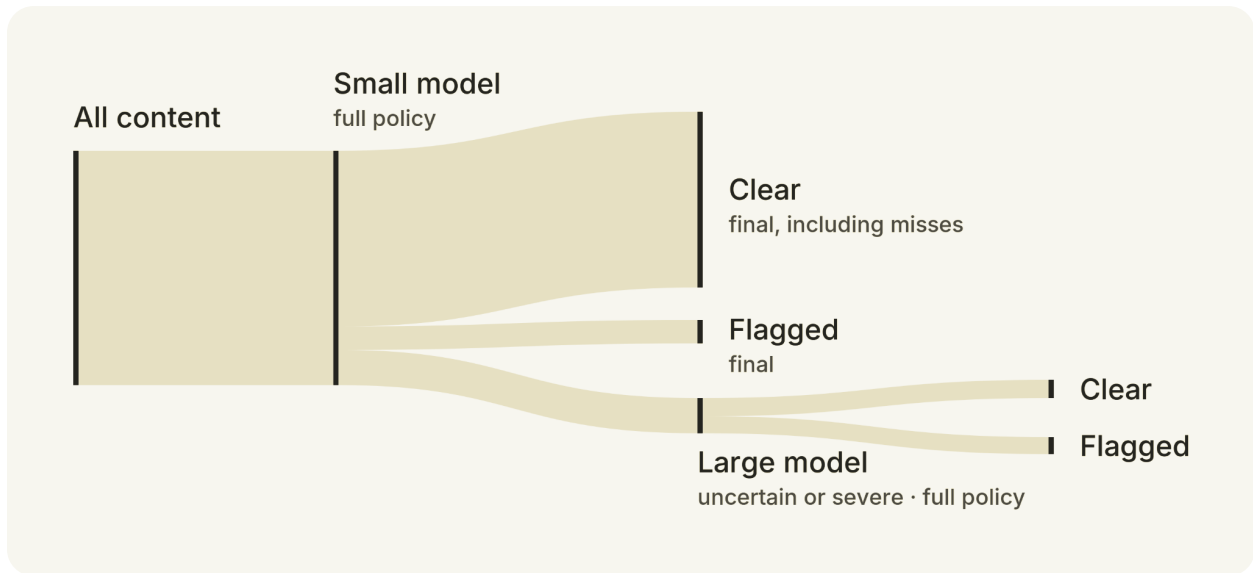
A small model decides only whether to flag or clear content. Flagged items go to a larger model with the full policy. In Musubi's testing, this was much cheaper than using the large model on everything and better at reporting every violation on items containing several.



**Figure 1.** Screen then escalate

## Small model with escalation for uncertainty

A small model applies the full policy to all content; only uncertain or severe items escalate. This was also much cheaper and caught slightly more, but the small model's confident decisions are final, including its misses.



**Figure 2.** Small model with escalation for uncertainty

## Label then judge

A fast labeler produces labels rather than decisions. If no label triggers, automatically approve the content. If a label triggers, a policy-aware reasoning model reads the labels, policy, and context. One label set can feed several policies.

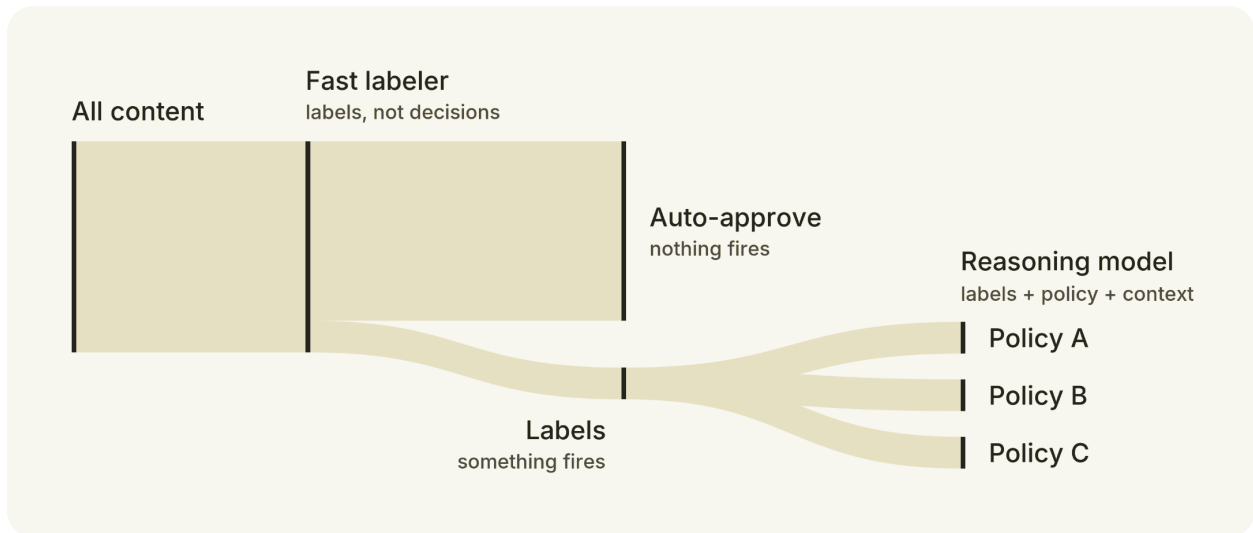


Figure 3. Label then judge

## Router policy followed by specialist policies

Use the same model family and route between prompts rather than models. Tune specialist policies for hard subcategories. Preserve signals across each handoff: if a specialist returns “clear,” do not discard other legitimate flags raised by the router.

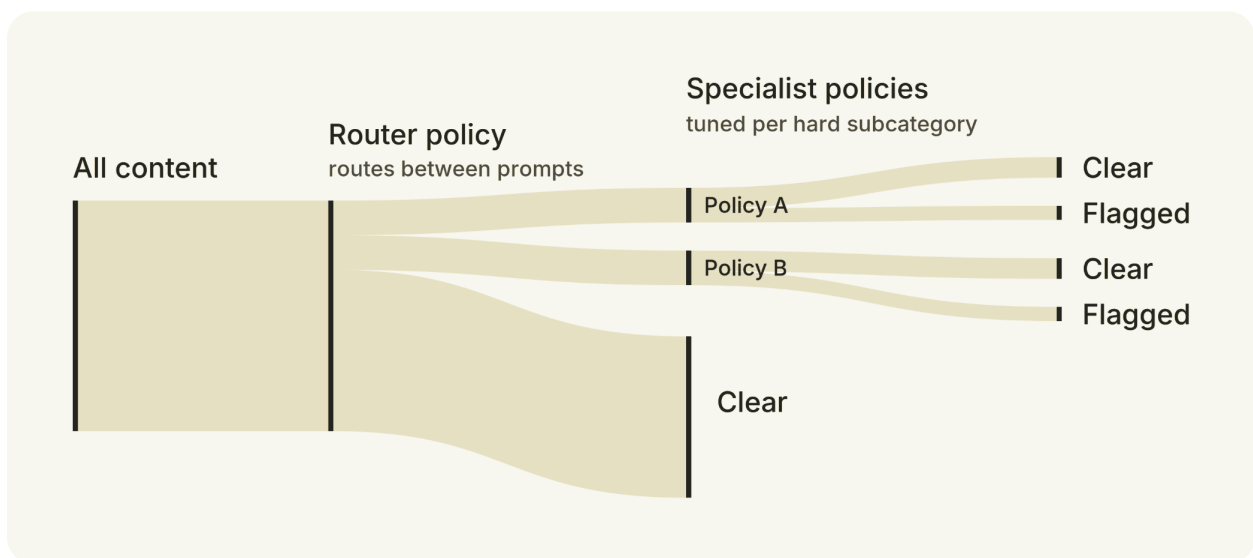


Figure 4. Router policy followed by specialist policies

## Combining architectures: live chat

These are building blocks rather than finished products. In production, these workflows usually combine two or three of them, with a deterministic precheck in front and human review at the end. Live chat is the clearest example, because its latency budget forces every choice.

For a single-model live workflow, prioritize latency, then balanced accuracy and per-language calibration. A stack might include:

- A deterministic keyword or hash precheck for known strings.
- One fast, policy-aware classifier that makes the decision using per-category thresholds, such as PolicyLM-1.7B.
- A larger model for decisions requiring a rationale, such as appeals, and as an LLM-as-a-judge on sampled production data to assess drift.

Pair this with Roblox's PII classifier for personal information and off-platform steering, and with Sentinel across users' recent messages to identify rare, severe patterns. Route those patterns to investigators rather than taking automatic action.

That stack is the small model with escalation for uncertainty, with a deterministic precheck in front and the escalation moved off the critical path. The same four blocks recombine differently on every other surface, as Appendix 1 shows.

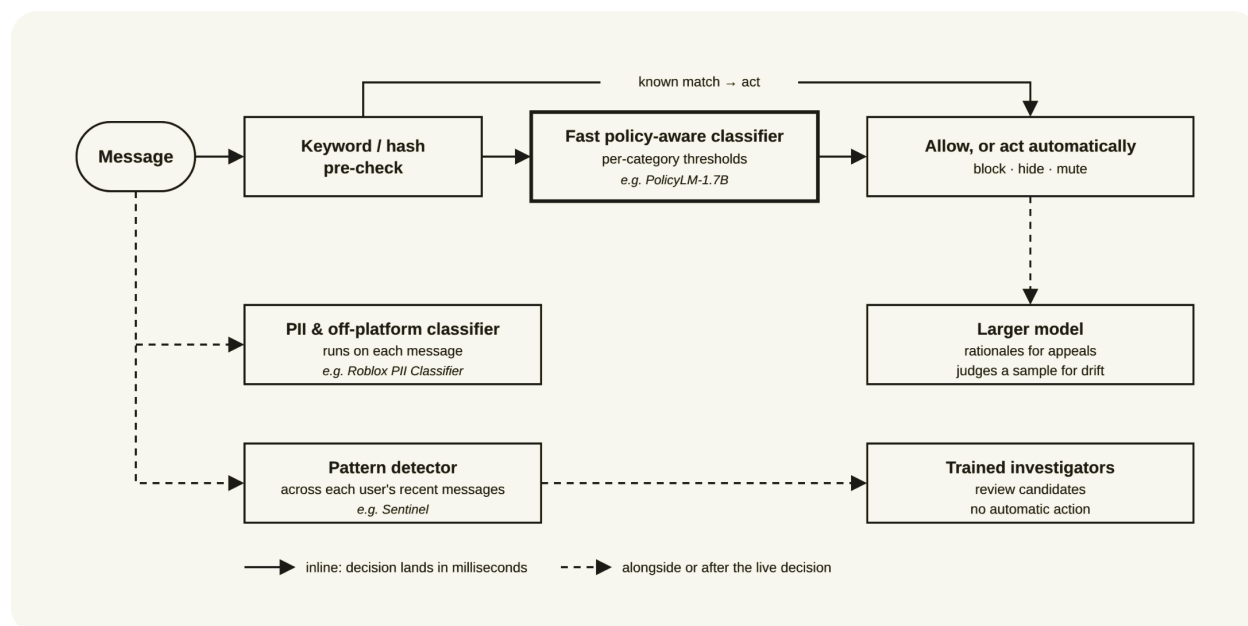


Figure 5. Live chat and real time text

# Conclusion

Trust & Safety enforcement used to be constrained by what a model had been trained on. With policy-steerable models, outcomes depend on the model, the policy text, and how they are connected. This gives T&S teams influence over the whole system as well as the policy document, but adds complexity and potential failure points.

Start simple, and add complexity when cost or latency requires it. Test early and often with golden datasets and production data so you understand what is changing and why. Spend model budget where it meaningfully improves outcomes. The largest model is not needed everywhere; small, open-source models are often sufficient.

AI changes quickly, which is why this report keeps its guidance as general and broadly applicable as possible. Model-specific findings appear as illustrative examples: results vary by model, policy, and platform, and no single team's testing can represent every platform.

These rapid changes make collaboration across platforms and organizations increasingly valuable. The best insights come from practitioners running models and workflows on real content and comparing notes. When one team identifies and publishes a new pattern, others can catch it sooner. We encourage practitioners to join us and share what they are finding.

If you want to test with us, bring your own policy to our upcoming workshop “Does a Model Follow Your Rules?” on Oct 27 and run it across ROOST Model Community models, including PolicyLM.

# Appendix 1: Examples of Routing Architecture

## User-uploaded images (label, then judge)

What matters most here is first-stage recall and cost. Your stack might look like this:

- A fast, high-recall image labeler or classifier runs on everything
- If nothing fires, auto-approve.
- If anything fires, a policy-aware LLM reads a more detailed policy, the labels, and the context, and makes a final decision.

**From the RMC, you could use** Shieldstral as the first pass, with a recall-tuned threshold. It asks one policy question per call, so keep the first-stage label set small and let the second stage do the detailed policy. Use gpt-oss-safeguard as the second stage when you want the reasoning logged.

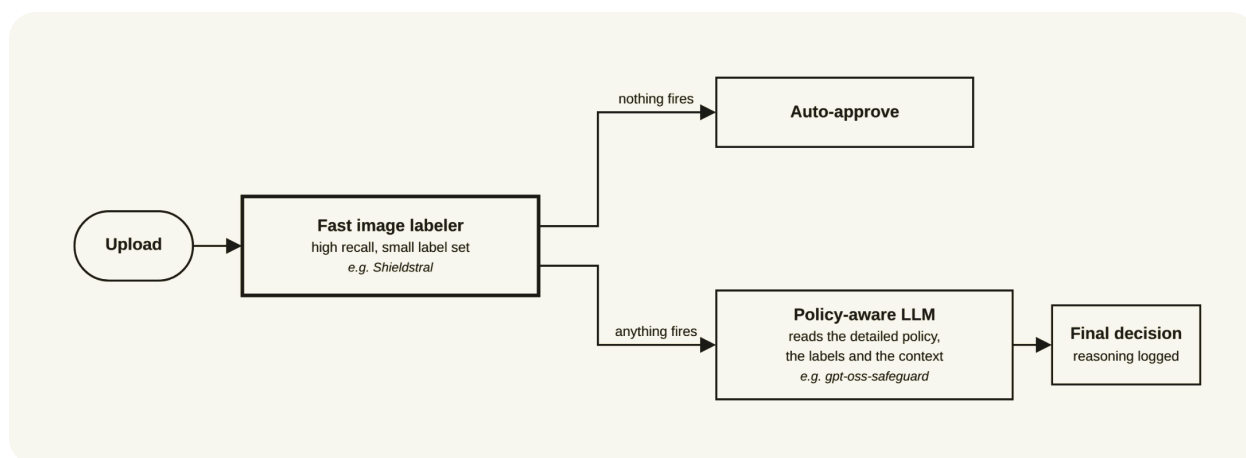
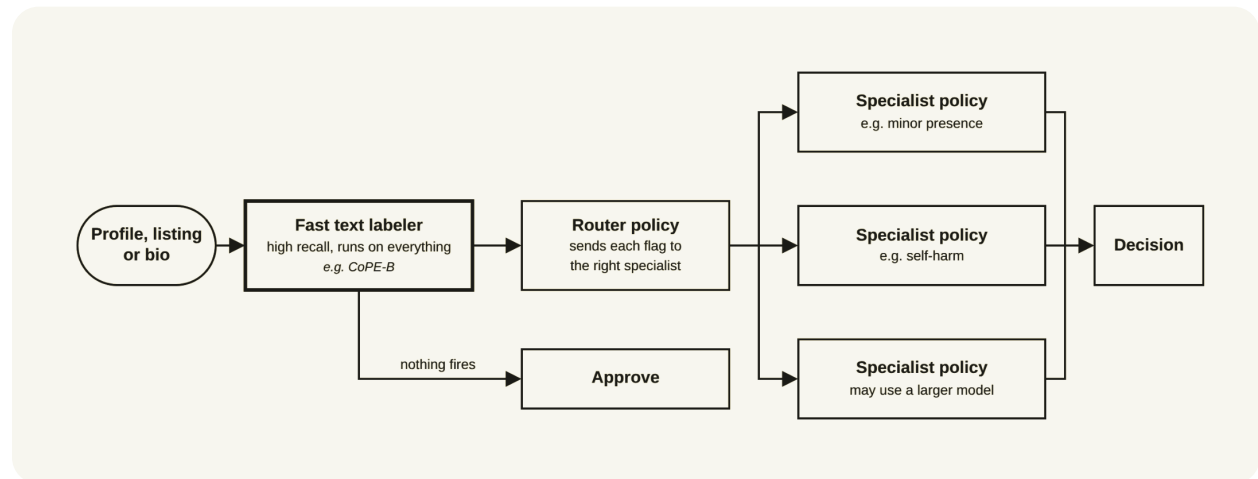


Figure 6. User-uploaded images

## Async review: profiles, listings, bios (accuracy-first)

What matters most here is accuracy and steerability, because policies change and specialists get tuned separately. Your stack might look like this:

- A fast, high-recall text labeler runs on everything (e.g. CoPE)
- A router policy sends hard subcategories to specialist policies, potentially with larger models attached



**Figure 7.** Async review: profiles, listings, bios

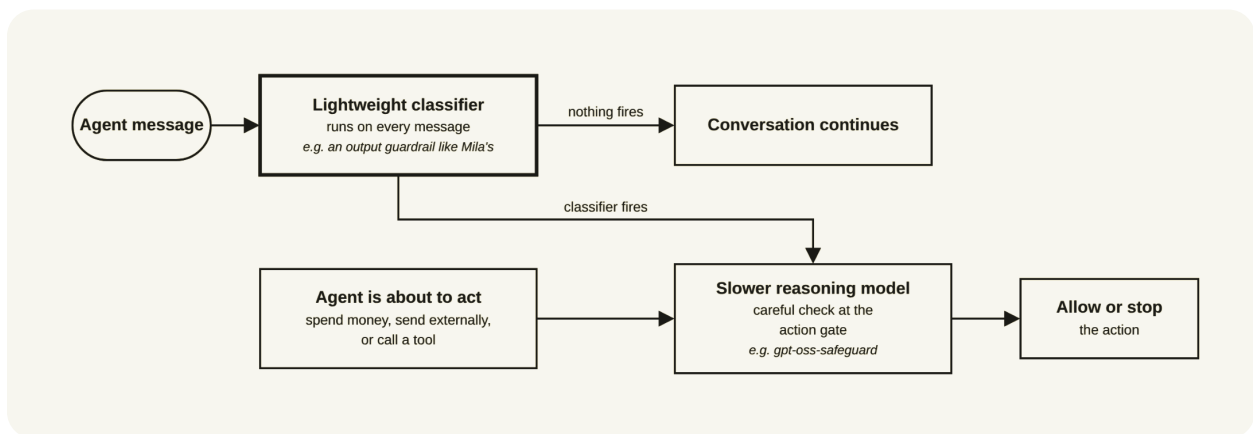
## Agent-to-agent safety

What matters most here is staying cheap while nothing is happening, and being careful at the moment of consequence. Your stack might look like this:

- A lightweight classifier on every message in the agent transcript.
- A slower reasoning model that runs only when the classifier fires, or when the receiving agent is about to take a real-world action (spend money, send content externally, call a tool).

**From the RMC, you could run** an output guardrail like Mila's on every assistant response, and use gpt-oss-safeguard as the careful check at the action gate.

An open question is what steerability means here, when the "policy" includes procedural rules.



**Figure 8.** Agent-to-agent safety

## Voice chat

What matters most here is latency and coverage across languages. Your stack might look like this:

- An audio classifier on voice segments (e.g. Roblox's Voice-Safety-Classifer-v3: segments up to 15 seconds, operating points per language).
- Flagged segments go to a transcript plus a text policy model, or to a human, for anything your policy defines beyond the classifier's fixed categories.

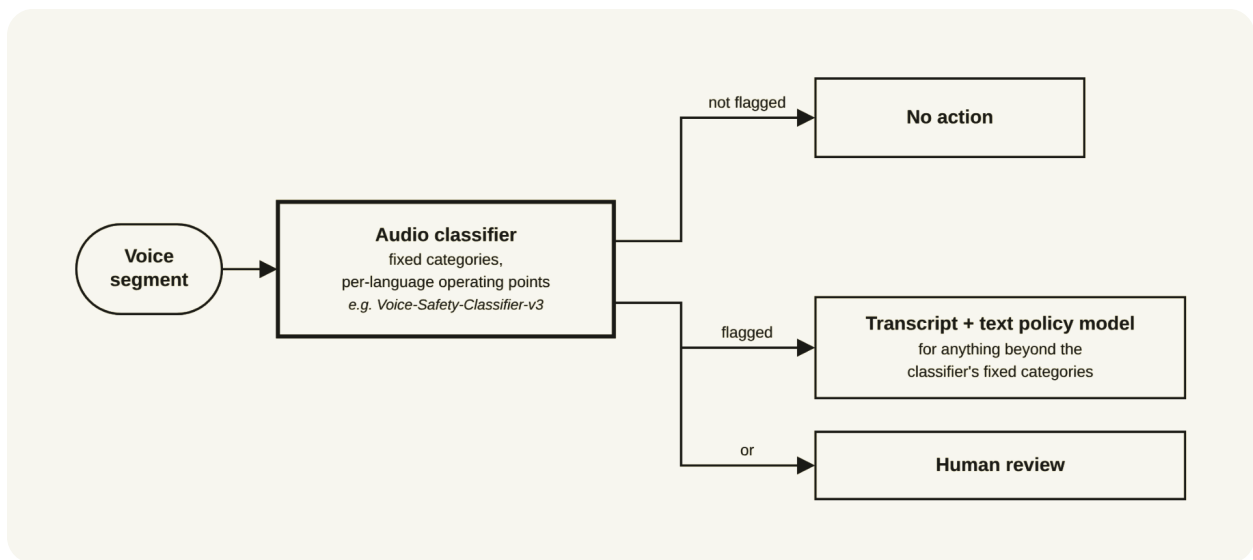
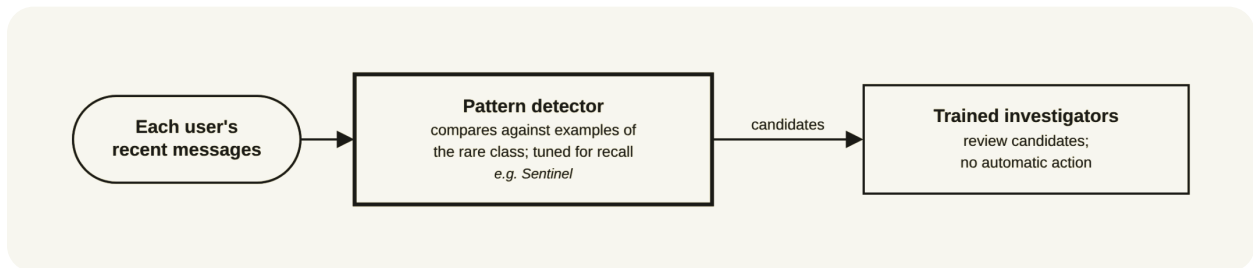


Figure 9. Voice chat

## Rare, severe harms (e.g. grooming patterns)

What matters most here is recall. Examples are scarce, and single messages often look harmless. Your stack might look like this:

- A pattern detector across each user's recent messages (e.g. Sentinel) generates candidates.
- Trained investigators review them. This tier finds cases; it doesn't take automatic action.



**Figure 10.** Rare, severe harms

## Appendix 2: Additional Routing Model Examples

### Deterministic precheck followed by a model

Hash, URL, or keyword matches block content before a model runs. This is inexpensive and fast, and sensitive matched content never reaches a third-party model.

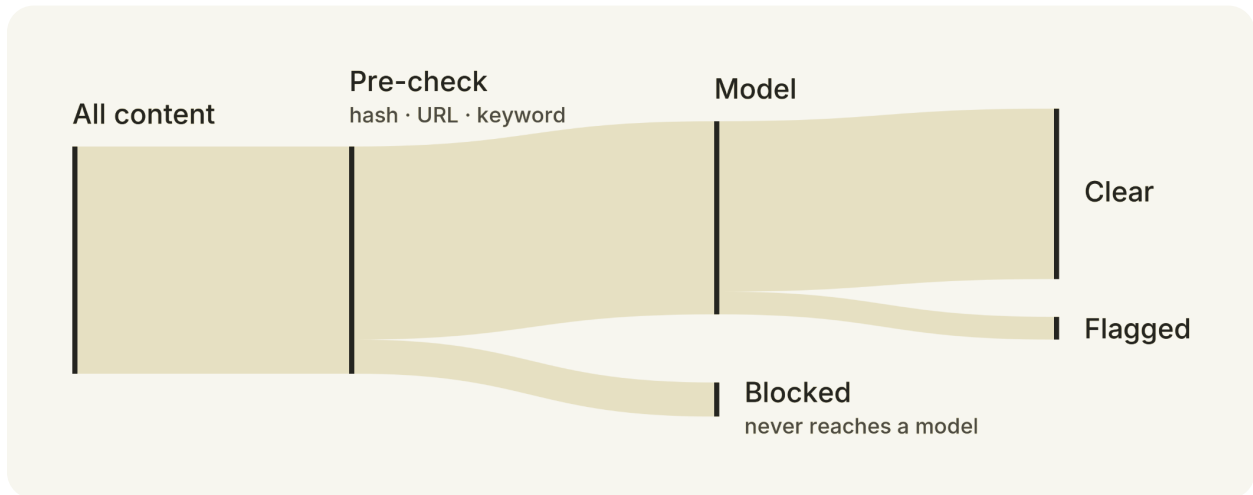


Figure 11. Deterministic precheck followed by a model

### Classifier with a scoring head

Add a simple scoring head to a classifier that returns scores for multiple categories. This converts scores into decisions without individual category thresholds and improves performance by accounting for interactions across categories, such as when adding profanity to teasing should escalate it to bullying.

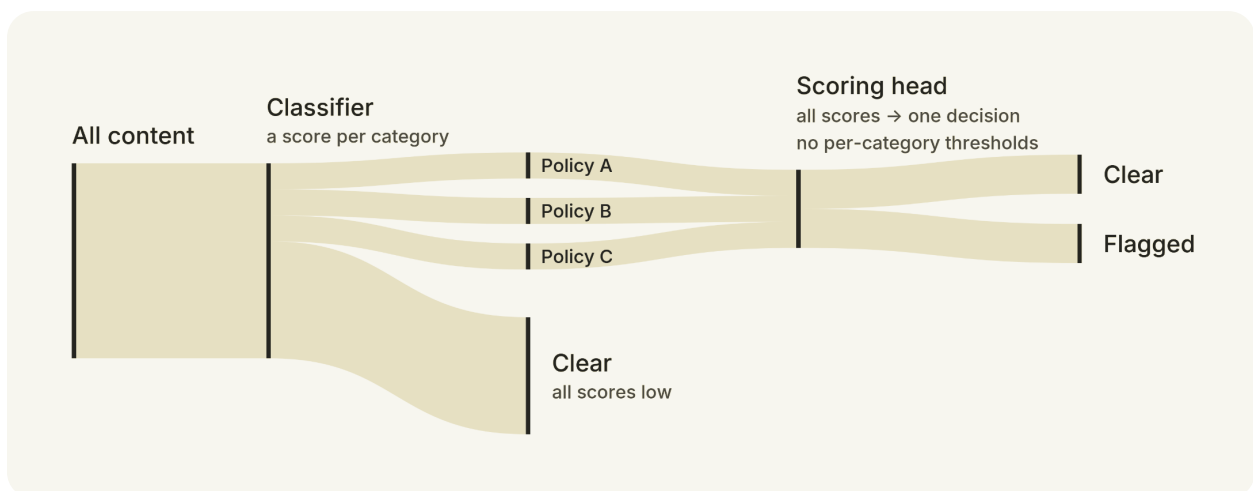


Figure 12. Classifier with a scoring head

## One check per category

Run a small yes/no policy for each category in parallel. In our testing, this caught the vast majority of violations, but each check saw only its own category and therefore overflagged safe content. It also cost more and increased the number of policies to maintain.

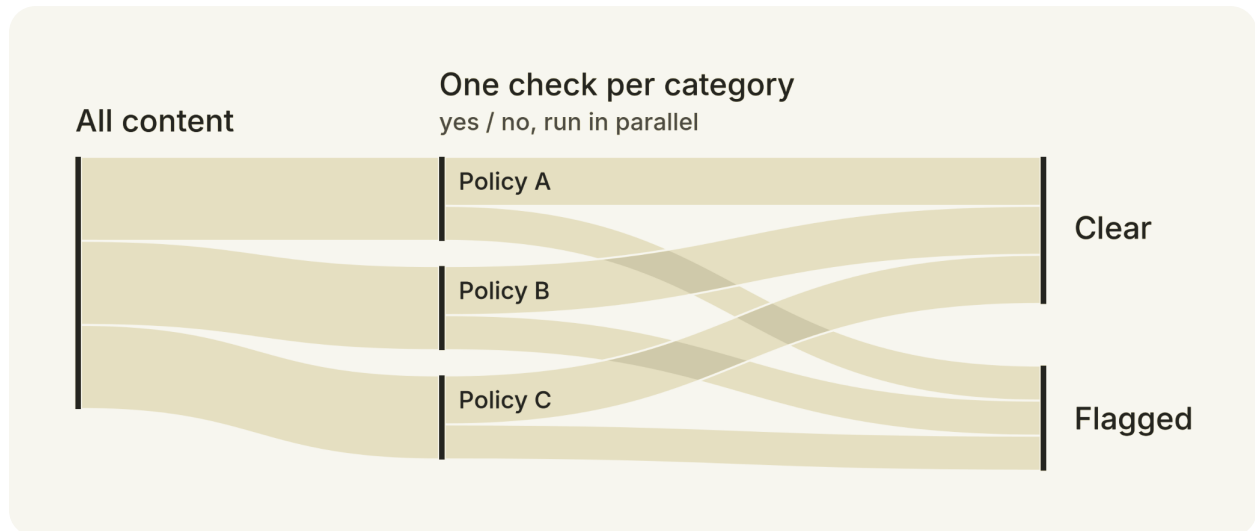


Figure 13. One check per category

## Behavioral signals followed by content review

A model using structured signals handles straightforward decisions. Ambiguous cases go to a content model or a human.

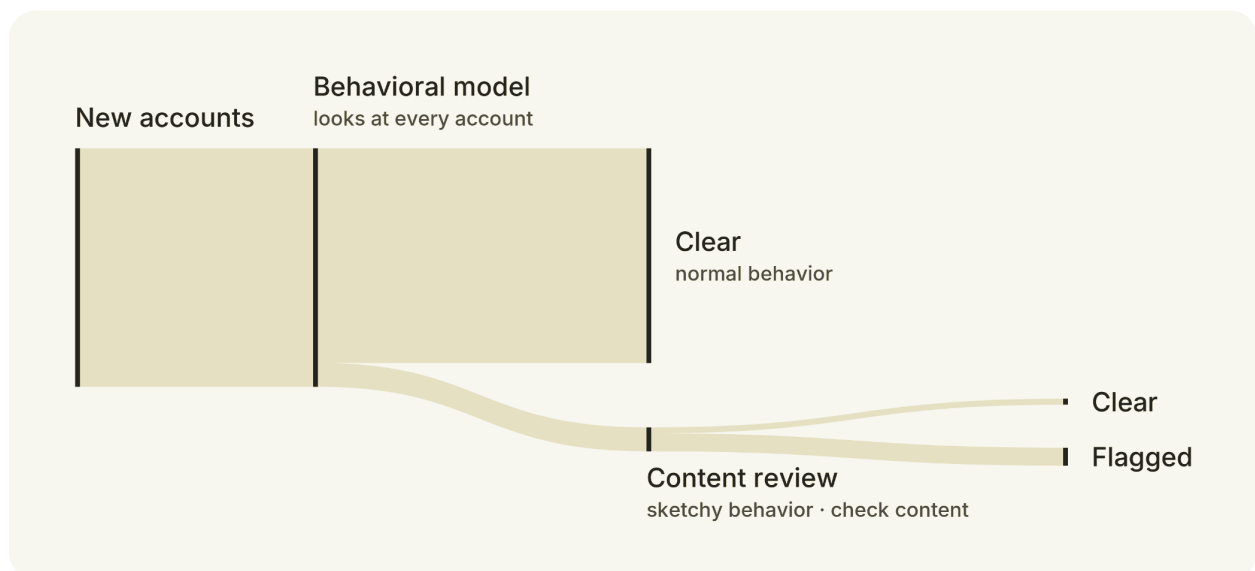


Figure 14. Behavioral signals followed by content review

## Content review followed by behavioral signals

A content model identifies potentially scam-related or suspicious content, then routes it to a larger model for a holistic review of the account and its behavior.

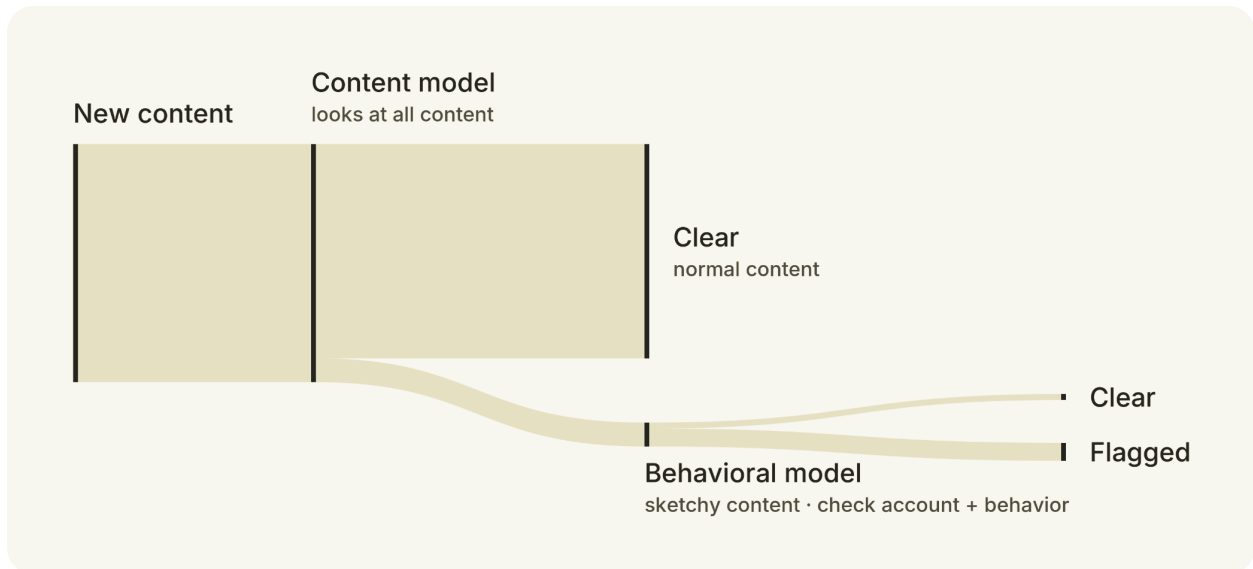


Figure 15. Content review followed by behavioral signals

## Provider fallback

Use a primary model, with a secondary model as fallback when an error, timeout, or quota limit occurs.

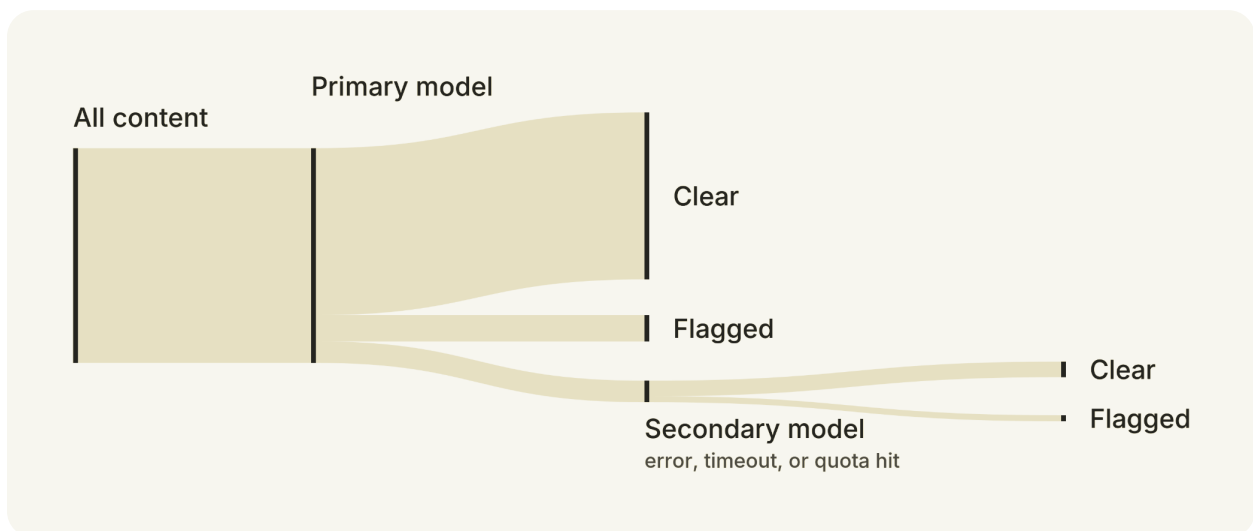
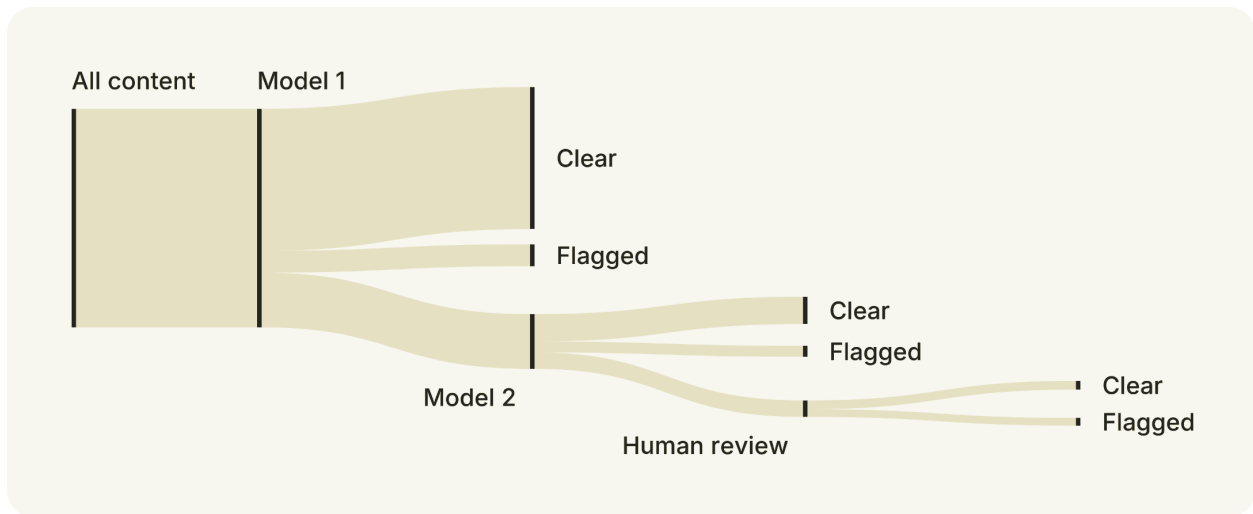


Figure 16. Provider fallback

## Human review as the final tier

Stacking models for straightforward decisions makes human judgment affordable at scale.



**Figure 17.** Human review as the final tier